

Motion Detector (M5Stack) Program

MotionDetector_AdjustGyroscope_M5_20180606

```
// BLE SERVER
//=====
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <M5Stack.h>
#include "utility/MPU9250.h"
#include "utility/quaternionFilters.h"
#include "utility/Button.h"

#define processing_out false
#define AHRS true // Set to false for basic data read
#define SerialDebug true // Set to true to get Serial output for debugging
#define LCD

#define SU "0000fff0-0000-1000-8000-00805f9b34fb"
#define CU "beb5483e-36e1-4688-b7f5-ea07361b26a8"
MPU9250 IMU;
String msg = "C";
String oldmsg = "C";
int thresholdValue = 30;

BLEService *pService;
BLEAdvertising *pAdvertising;
BLECharacteristic *pCharacteristic;
//=====

void setup() {
  //Serial.begin(115200);
  MS.begin();
  Wire.begin();
  MS.Lcd.fillScreen(BLACK);

  //-----
  BLEDevice::init("MASTER");
  BLEServer *pServer = BLEDevice::createServer();
  pService = pServer->createService(SU);
  pCharacteristic = pService->createCharacteristic(
    CU, BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE);
  pCharacteristic->setValue(msg.c_str());
  pService->start();
  pAdvertising = pServer->getAdvertising();
  pAdvertising->addServiceUUID(pService->getUUID());
  pAdvertising->start();

  IMU.initMPU9250();
  // Initialize device for active mode read of accelerometer, gyroscope, and
  // temperature
  Serial.println("MPU9250 initialized for active data mode....");

  MS.Lcd.setTextSize(1);
  MS.Lcd.setTextColor(GREEN, BLACK);
  MS.Lcd.fillScreen(BLACK);
}
//=====

void loop() {

  // If intPin goes high, all data registers have new data
  // On interrupt, check if data ready interrupt
  if (IMU.readByte(MPU9250_ADDRESS, INT_STATUS) & 0x01)
  {
    IMU.readGyroData(IMU.gyroCount); // Read the x/y/z adc values
    IMU.getGres();

    // Calculate the gyro value into actual degrees per second

    // This depends on scale being set
    IMU.gx = (float)IMU.gyroCount[0]*IMU.gRes;
    IMU.gy = (float)IMU.gyroCount[1]*IMU.gRes;
    IMU.gz = (float)IMU.gyroCount[2]*IMU.gRes;
  } // if (readByte(MPU9250_ADDRESS, INT_STATUS) & 0x01)

  // Must be called before updating quaternions!
  IMU.updateTime();

  // Sensors x (y)-axis of the accelerometer is aligned with the y (x)-axis of
  // the magnetometer; the magnetometer z-axis (+ down) is opposite to z-axis
  // (+ up) of accelerometer and gyro! We have to make some allowance for this
  // orientation mismatch in feeding the output to the quaternion filter. For the
  // MPU-9250, we have chosen a magnetic rotation that keeps the sensor forward
  // along the x-axis just like in the LSM9DS0 sensor. This rotation can be
  // modified to allow any convenient orientation convention. This is ok by
  // aircraft orientation standards! Pass gyro rate as rad/s
  // MadgwickQuaternionUpdate(ax, ay, az, gx*PI/180.0f, gy*PI/180.0f, gz*PI/180.0f, mx, my, mz);
  MahonyQuaternionUpdate(IMU.ax, IMU.ay, IMU.az, IMU.gx*DEG_TO_RAD,
    IMU.gy*DEG_TO_RAD, IMU.gz*DEG_TO_RAD, IMU.mx,
    IMU.my, IMU.mz, IMU.deltat);

  if (!AHRS)
  {
    IMU.delt.t = millis() - IMU.count;
    if (IMU.delt.t > 500)
    {
      if(SerialDebug)
      {
        // Print gyro values in degree/sec
        Serial.print("X-gyro rate: "); Serial.print(IMU.gx, 3);
        Serial.print(" degrees/sec ");
        Serial.print("Y-gyro rate: "); Serial.print(IMU.gy, 3);
        Serial.print(" degrees/sec ");
        Serial.print("Z-gyro rate: "); Serial.print(IMU.gz, 3);
        Serial.print(" degrees/sec ");
        Serial.println(" degrees/sec");
      }
    }
  }

  #ifdef LCD
    MS.Lcd.fillScreen(BLACK);
    MS.Lcd.setTextColor(GREEN, BLACK);
    MS.Lcd.setCursor(0, 0); MS.Lcd.print("MPU9250/AK8963");
    MS.Lcd.setCursor(0, 32); MS.Lcd.print(" x y z ");

    MS.Lcd.setCursor(0, 64); MS.Lcd.print((int)(IMU.gx));
    MS.Lcd.setCursor(32, 64); MS.Lcd.print((int)(IMU.gy));
    MS.Lcd.setCursor(64, 64); MS.Lcd.print((int)(IMU.gz));
    MS.Lcd.setCursor(96, 64); MS.Lcd.print("a/s");

  #endif // LCD

  IMU.count = millis();
  // digitalWrite(myLed, !digitalRead(myLed)); // toggle led
  } // if (IMU.delt.t > 500)
} // if (AHRS)
else
{
  // Serial print and/or display at 0.5 s rate independent of data rates
  IMU.delt.t = millis() - IMU.count;

  // update LCD once per half-second independent of read rate
  // if (IMU.delt.t > 500)
  if (IMU.delt.t > 100)
  {
    if(SerialDebug)
    {

```

```
Serial.print("gx = "); Serial.print( IMU.gx, 2);
Serial.print(" gy = "); Serial.print( IMU.gy, 2);
Serial.print(" gz = "); Serial.print( IMU.gz, 2);
Serial.println(" deg/s");
}

if(IMU.gz>thresholdValue && (oldmsg != "A")){
  msg = "A";
  oldmsg = msg;
  pCharacteristic->setValue(msg.c_str());
}
else{
  if(IMU.gz>=-thresholdValue){
    msg = oldmsg;
    pCharacteristic->setValue(msg.c_str());
  }
  else{
    msg = "C";
    oldmsg = msg;
    pCharacteristic->setValue(msg.c_str());
  }
}

#ifdef LCD
  // MS.Lcd.fillScreen(BLACK);
  MS.Lcd.setCursor(0, 0); MS.Lcd.print(" x y z ");

  MS.Lcd.setCursor(0, 44);
  MS.Lcd.printf("% 6d % 6d % 6d o/s \r\n", (int)(IMU.gx), (int)(IMU.gy), (int)(IMU.gz));

  MS.Lcd.setCursor(0, 180);
  MS.Lcd.printf("Threshold: % 6d \r\n", (thresholdValue));
  MS.Lcd.print(" Adjust - + ");
}

IMU.count = millis();
IMU.sumCount = 0;
IMU.sum = 0;

} // if (IMU.delt.t > 500)
} // if (AHRS)

if (digitalRead(38)) {
  thresholdValue += 1;
  if (thresholdValue > 80){thresholdValue = 80;}
  delay(150);
}

if (digitalRead(37)) {
  thresholdValue -= 1;
  if (thresholdValue < 1){thresholdValue = 1;}
  delay(150);
}
}
```